25-Context容器（中）：Tomcat如何隔离Web应用？

我在专栏上一期提到，Tomcat通过自定义类加载器WebAppClassLoader打破了双亲委托机制，具体来说就是重写了JVM的类加载器ClassLoader的findClass方法和loadClass方法，这样做的目的是优先加载Web应用目录下的类。除此之外，你觉得Tomcat的类加载器还需要完成哪些需求呢？或者说在设计上还需要考虑哪些方面？

我们知道，Tomcat作为Servlet容器，它负责加载我们的Servlet类，此外它还负责加载Servlet所依赖的JAR包。并且Tomcat本身也是也是一个Java程序，因此它需要加载自己的类和依赖的JAR包。首先让我们思考这一下这几个问题：

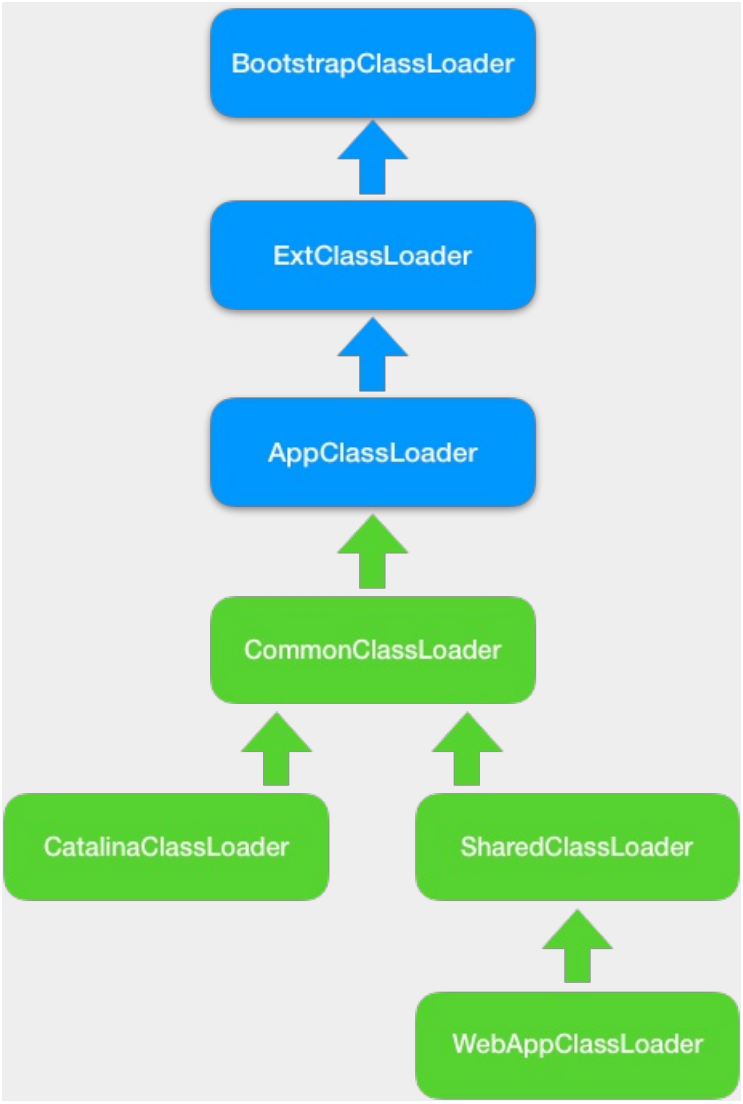
1. 假如我们在Tomcat中运行了两个Web应用程序，两个Web应用中有同名的Servlet，但是功能不同，Tomcat需要同时加载和管理这两个同名的Servlet类，保证它们不会冲突，因此Web应用之间的类需要隔离。
2. 假如两个Web应用都依赖同一个第三方的JAR包，比如Spring，那Spring的JAR包被加载到内存后，Tomcat要保证这两个Web应用能够共享，也就是说Spring的JAR包只被加载一次，否则随着依赖的第三方JAR包增多，JVM的内存会膨胀。
3. 跟JVM一样，我们需要隔离Tomcat本身的类和Web应用的类。

在了解了Tomcat的类加载器在设计时要考虑的这些问题以后，今天我们主要来学习一下Tomcat是如何通过设计多层次的类加载器来解决这些问题的。

Tomcat类加载器的层次结构

为了解决这些问题，Tomcat设计了类加载器的层次结构，它们的关系如下图所示。下面我来详细解释为什么要设计这些类加载器，告诉你它们是怎么解决上面这些问题的。

仅供个人学习 请勿传播 用户标记 mannix小查理



我们先来看**第1个问题**，假如我们使用JVM默认AppClassLoader来加载Web应用，AppClassLoader只能加载一个Servlet类，在加载第二个同名Servlet类时，AppClassLoader会返回第一个Servlet类的Class实例，这是因为在AppClassLoader看来，同名的Servlet类只被加载一次。

因此Tomcat的解决方案是自定义一个类加载器WebAppClassLoader， 并且给每个Web应用创建一个类加载器实例。我们知道，Context容器组件对应一个Web应用，因此，每个Context容器负责创建和维护一个WebAppClassLoader加载器实例。这背后的原理是，**不同的加载器实例加载的类被认为是不同的类**，即使它们的类名相同。这就相当于在Java虚拟机内部创建了一个个相互隔离的Java类空间，每一个Web应用都有自己的类空间，Web应用之间通过各自的类加载器互相隔离。

SharedClassLoader

我们再来看**第2个问题**，本质需求是两个Web应用之间怎么共享库类，并且不能重复加载相同的类。我们知道，在双亲委托机制里，各个子加载器都可以通过父加载器去加载类，那么把需要共享的类放到父加载器的加载路径下不就行了吗，应用程序也正是通过这种方式共享JRE的核心类。因此Tomcat的设计者又加了一个类加载器SharedClassLoader，作为WebAppClassLoader的父加载器，专门来加载Web应用之间共享的类。如果WebAppClassLoader自己没有加载到某个类，就会委托父加载器SharedClassLoader去加载这个类，SharedClassLoader会在指定目录下加载共享类，之后返回给WebAppClassLoader，这样共享的问题就解决了。

CatalinaClassloader

我们来看**第3个问题**，如何隔离Tomcat本身的类和Web应用的类？我们知道，要共享可以通过父子关系，要隔离那就需要兄弟关系了。兄弟关系就是指两个类加载器是平行的，它们可能拥有同一个父加载器，但是两个兄弟类加载器加载的类是隔离的。基于此Tomcat又设计一个类加载器CatalinaClassloader，专门来加载Tomcat自身的类。这样设计有个问题，那Tomcat和各Web应用之间需要共享一些类时该怎么办呢？

CommonClassLoader

老办法，还是再增加一个CommonClassLoader，作为CatalinaClassloader和SharedClassLoader的父加载器。CommonClassLoader能加载的类都可以被CatalinaClassLoader和SharedClassLoader 使用，而CatalinaClassLoader和SharedClassLoader能加载的类则与对方相互隔离。WebAppClassLoader可以使用SharedClassLoader加载到的类，但各个WebAppClassLoader实例之间相互隔离。

Spring的加载问题

在JVM的实现中有一条隐含的规则，默认情况下，如果一个类由类加载器A加载，那么这个类的依赖类也是由相同的类加载器加载。比如Spring作为一个Bean工厂，它需要创建业务类的实例，并且在创建业务类实例之前需要加载这些类。Spring是通过调用`Class.forName`来加载业务类的，我们来看一下forName的源码：

```
public static Class<?> forName(String className) {
    Class<?> caller = Reflection.getCallerClass();
    return forName0(className, true, ClassLoader.getClassLoader(caller), caller);
}
```

可以看到在forName的函数里，会用调用者也就是Spring的加载器去加载业务类。

我在前面提到，Web应用之间共享的JAR包可以交给SharedClassLoader来加载，从而避免重复加载。Spring作为共享的第三方JAR包，它本身是由SharedClassLoader来加载的，Spring又要去加载业务类，按照前面那条规则，加载Spring的类加载器也会用来加载业务类，但是业务类在Web应用目录下，不在SharedClassLoader的加载路径下，这该怎么办呢？

于是线程上下文加载器登场了，它其实是一种类加载器传递机制。为什么叫作“线程上下文加载器”呢，因为这个类加载器保存在线程私有数据里，只要是同一个线程，一旦设置了线程上下文加载器，在线程后续执行过程中就能把这个类加载器取出来用。因此Tomcat为每个Web应用创建一个WebAppClassLoarder类加载器，并在启动Web应用的线程里设置线程上下文加载器，这样Spring在启动时就将线程上下文加载器取出来，用来加载Bean。Spring取线程上下文加载的代码如下：

```
cl = Thread.currentThread().getContextClassLoader();
```

本期精华

今天我介绍了JVM的类加载器原理并剖析了源码，以及Tomcat的类加载器的设计。重点需要你理解的是，Tomcat的Context组件为每个Web应用创建一个WebAppClassLoarder类加载器，由于**不同类加载器实例加**

仅供个人学习 请勿传播 用户标记 mannix小童鞋

载的类是互相隔离的，因此达到了隔离Web应用的目的，同时通过CommonClassLoader等父加载器来共享第三方JAR包。而共享的第三方JAR包怎么加载特定Web应用的类呢？可以通过设置线程上下文加载器来解决。而作为Java程序员，我们应该牢记的是：

- 每个Web应用自己的Java类文件和依赖的JAR包，分别放在WEB-INF/classes和WEB-INF/lib目录下面。
- 多个应用共享的Java类文件和JAR包，分别放在Web容器指定的共享目录下。
- 当出现ClassNotFound错误时，应该检查你的类加载器是否正确。

线程上下文加载器不仅仅可以用在Tomcat和Spring类加载的场景里，核心框架类需要加载具体实现类时都可以用到它，比如我们熟悉的JDBC就是通过上下文类加载器来加载不同的数据库驱动的，感兴趣的话可以深入了解一下。

课后思考

在StandardContext的启动方法里，会将当前线程的上下文加载器设置为WebAppClassLoader。

```
originalClassLoader = Thread.currentThread().getContextClassLoader();
Thread.currentThread().setContextClassLoader(webApplicationClassLoader);
```

在启动方法结束的时候，还会恢复线程的上下文加载器：

```
Thread.currentThread().setContextClassLoader(originalClassLoader);
```

这是为什么呢？

不知道今天的内容你消化得如何？如果还有疑问，请大胆的在留言区提问，也欢迎你把你的课后思考和心得记录下来，与我和其他同学一起讨论。如果你觉得今天有所收获，欢迎你把它分享给你的朋友。



精选留言：

- nightmare 2019-07-06 08:18:32
  老师，上下文加载器是不是比如说我在加载spring的线程设置为webappclassloader那么就算spring的jar是由shared classloader加载的，那么spring加载的过程中也是由webappclassloader来加载，而用完设置回去，是因为我只需要跨classloader的时候才需要线程上下文加载器 [1赞]

  作者回复2019-07-06 22:17:12
  是的👍

- 一颗苹果 2019-07-07 19:11:01
  老师请问下，如果tomcat的不同应用引用了不同版本的spring依赖，sharedClassloader 怎么区分不同版本呢

  作者回复2019-07-07 22:25:11
  这种情况就不是公共类库了，应该放到各Web应用的路径下去

- 玉芟 2019-07-07 02:11:36
  老师，您好：
  我对Thread.currentThread().setContextClassLoader(ClassLoader cl)用法一直有个疑问：
  - setContextClassLoader以后是不是只能显示地通过getContextClassLoader获取ClassLoader后调用loadClass(String name, boolean resolve)方法加载类才能是自定义加载器加载的(验证方法：打印obj.getClass().getClassLoader())？
  - setContextClassLoader以后通过Class.forName(String name)方法等反射得到的类是不是就只能是AppClassLoader加载的？
  我做了个实验：
  自定义类加载器：
  public class DIYClassLoader extends URLClassLoader {
  public DIYClassLoader(URL[] urls) { super(urls); }
  /**
  * 策略很简单：
  * 1)、首先尝试ExtClassLoader|BootstrapClassLoader加载

仅供个人学习 请勿传播 用户标记 mannix小童鞋

```
* 2)、之后尝试自己加载
* 3)、最后尝试真正父加载器加载
*/
@Override
protected Class<?> loadClass(String name, boolean resolve) throws ClassNotFoundException {
Class<?> c = findLoadedClass(name);
ClassLoader parent = getParent();
if (parent != null) {
ClassLoader ecl = parent;
while (ecl.getParent() != null)// 找ExtClassLoader
ecl = ecl.getParent();
try {
c = ecl.loadClass(name);
} catch (ClassNotFoundException e) { }
if (c == null) {
try {
c = findClass(name);// DIYClassLoader自己来
} catch (ClassNotFoundException e) {}
if (c == null) {
// 尝试真正父加载器加载，多半是AppClassLoader
c = parent.loadClass(name);
}
}
}else {
// 直接自己尝试加载
c = findClass(name);
}
if (resolve)
resolveClass(c);
return c;
}
}
```

main方法：

```
URL url = Main.class.getClassLoader().getResource(".");
DIYClassLoader scl = new DIYClassLoader(new URL[] {url});
Thread.currentThread().setContextClassLoader(scl);
Class clazz = Class.forName("xx.xx.Xxx");
// sun.misc.Launcher$AppClassLoader@18b4aac2
clazz = scl.loadClass("xx.xx.Xxx");
// xx.xx.DIYClassLoader@682a0b20
```

不知道我把问题描述清楚了吗？还望老师解答

作者回复2019-07-07 22:27:43

线程上下文加载器本质是线程私有数据，需要显式拿出来，调getContextClassLoader拿

- nightmare 2019-07-06 23:14:02

  老师我今天做了试验，在tomcat下和conf同级建立shared目录，然后把两个项目的spring的jar包放到shared目录下，然后webapp/class下的spring的jar包删除，启动报找不到spring的jar包，tomcat版本为7.x，是不是还需要配置什么啊，请老师帮忙指导一下

  作者回复2019-07-07 22:24:08

你可以在Tomcat conf目录下的catalina.properties文件中配置各加载器的加载路径

- 王之刚 2019-07-06 19:20:34
最后的问题没有想明白，有人能详细解释一下吗？

作者回复2019-07-06 21:43:32
线程上下文加载器其实是线程的一个私有数据，跟线程绑定的，这个线程做完启动Context组件的事情后，会被回收到线程池，之后被用来做其他事情，为了不影响其他事情，需要恢复之前的线程上下文加载器。

- 陆离 2019-07-06 18:14:53
项目有用到切换数据源。
先创建多个DataSource实例，然后put到AbstractRoutingDataSource的继承类targetDataSources这个map中，最后通过线程的threadlocal带的key值切换。
这个和今天讲的classloader 有关吗？

作者回复2019-07-06 21:44:55
跟线程上下文类加载器一样，threadlocal中的key也是线程私有数据。

- Cy190622 2019-07-06 16:01:20

老师好，您讲个很通透。还有一点问题请教一下：
1.线程上下文的加载器是不是指定子类加载器来加载具体的某个桥接类。比如JDBC的Driver的加载。
2.每个Web下面的java类和jar(WEB-INF/classes和WEB-INF/lib),都是WebAppClassLoader加载吗？
3.Web容器指定的共享目录一般是在什么路径下

作者回复2019-07-06 22:02:10
1和2你说的都准确。

CommonClassLoader对应<Tomcat>/common/*
CatalinaClassLoader对应 <Tomcat >/server/*
SharedClassLoader对应 <Tomcat >/shared/*
WebAppClassloader对应 <Tomcat >/webapps/<app>/WEB-INF/*目录

- 业余爱好者 2019-07-06 10:05:27
之前做了一个项目，在tomcat下面部署了两个springboot打的war，这两个war都依赖了同一个数据访问用的jar包，tomcat在启动第二个war项目时，报什么datasource已经实例化的一个错误，导致第二个项目启动失败。后来查了下资料，在application.yml里禁用了jmx解决。

虽然问题解决了，但却不明就里，不知道是不是web应用没有做隔离的缘故。不知道这样理解对不对。。

作者回复2019-07-06 22:11:53
应该在Tomcat安装目录下建一个shared目录，把web应用共享的库放这个目录下

- 大漠落木 2019-07-06 08:39:42
找不到 CommonClassLoader CatalinaClassLoader SharedClassLoader 这三个类

public class WebappClassLoader extends WebappClassLoaderBase

public abstract class WebappClassLoaderBase extends URLClassLoader

仅供个人学习 请勿传播 用户标记 mannix小查理

作者回复2019-07-06 22:16:37

前面三个是加载器实例名，不是类名，你可以在BootStrap.java中找到